

Name or Value? Cue Conflict Reveals When Chain of Thought Overrides Lexical Priors

Anonymous ACL submission

Abstract

Variable names that contradict a program’s behavior degrade the accuracy of language models, but a drop in accuracy does not indicate where inside the model the name interferes. Interpretability studies of multi-step arithmetic find that intermediate values are computed during chain-of-thought (CoT) generation; in those tasks, however, the name of a variable and its computed value never disagree. We introduce a cue-conflict version of such a task, in which the same problem is presented with congruent, neutral, and incongruent variable names, and evaluate eight models on five difficulty levels with three sets of few-shot examples. Without CoT, the models use the value denoted by the name: a congruent name raises accuracy by up to 20 points, and an incongruent name is produced as the answer up to 20 points above chance. With CoT, we observe no effect of the name’s value at any level, with one exception among the models we test. Linear probes indicate that the lure value is decodable at the name’s own tokens and is absent at the step where the chain writes the variable’s value, and activation patching localizes the no-CoT effect to the early layers at the equation that defines the name. Whether the chain states the value does not predict the size of the effect.

1 Introduction

Consider a short arithmetic problem written with named variables:

pen=1 + cup, cup=2 + 3; pen=?

The answer is 6. We then rename the second variable to the word two:

pen=1 + two, two=2 + 3; pen=?

The arithmetic is unchanged and the answer is still 6, but the name of the variable now denotes 2, while the variable itself holds the value 5. We refer to 2 as the *lure value* of the name and to 5 as its *true value*. If a model answers 2, or uses 2 in

a later step, the meaning of the name has reached the computation.

Placing two cues in conflict in this way is the design of the Stroop task (Stroop, 1935), in which the word RED is printed in blue ink, and of the shape–texture experiments on vision models (Geirhos et al., 2019). We apply this design to multi-step arithmetic with variable assignments.

Motivation. Names that contradict the surrounding code degrade model performance (Le et al., 2025; Lam et al., 2025; Le et al., 2026), but these results do not indicate *where* the name interferes, since a drop in accuracy is consistent with a corrupted input representation, with a corrupted computation, and with a correct computation that is misread at the output. Interpretability studies on clean tasks do not distinguish these cases either. Kudo et al. (2026) show, using linear probes and activation patching on arithmetic of this kind, that the value of each variable becomes linearly decodable only as the CoT writes it, but their variable names are single letters, so a decoded value of 5 can only originate from the computation. Our incongruent names supply a competing value, so that the output of a probe indicates which of the two sources the representation holds.

We construct matched sets of problems: the same arithmetic rendered in the naming conditions of Table 1 at the five difficulty levels of Table 2, solved with and without a written chain of thought, on eight models. Without CoT, the models use the value denoted by the name: a congruent name adds up to 20 points of accuracy, and an incongruent name is produced as the answer up to 20 points above chance. With CoT, we observe no effect of the name’s value at any level, with one exception among the models we test. Probes indicate that the lure value is decodable at the name’s own tokens and is absent at the step that writes the variable’s value.

Condition	Problem	True
Letter	A=1 + B, B=2 + 3; A=?	—
Neutral	pen=1 + cup, cup=2 + 3; pen=?	—
Congruent	pen=1 + five, five=2 + 3; pen=?	5
Incongruent	pen=1 + two, two=2 + 3; pen=?	5
Irrelevant (level 4)	pen=1 + cup, cup=2 + 3, two=4 + 5; pen=?	9

Table 1: The naming conditions on one problem, whose answer is 6 in every row of the top block. The model writes either the chain pen=1 + two, two=2 + 3, two=5, pen=1 + two, pen=1 + 5, pen=6 or the answer pen=6 alone. *True* is the value of the variable carrying the number word, and *Lure* is the value that word denotes. Each problem also has a twin with a different neutral word and a twin with a different lure, used as patching controls.

Level	Problem (neutral names)	Steps	Held
1	pen=1 + cup, cup=2; pen=?	1	1
2	pen=2 + 3, cup=1 + pen; cup=?	2	0
3	pen=1 + cup, cup=2 + 3; pen=?	2	1
4	pen=1 + cup, cup=2 + 3, box=4 + 5; pen=?	2	1
5	pen=1 + cup, cup=2 + box, box=1 + 2; pen=?	3	2

Table 2: The five levels of Kudo et al. (2026). *Steps* is the number of additions or subtractions required, *Held* the number of variables that cannot be resolved when first read, and *Distr.* the number of equations the answer does not depend on. The chain restates each equation and writes each variable’s value in turn, as in cup=2 + 3, cup=5, pen=1 + cup, pen=1 + 5, pen=6. At level 1 the intermediate value is stated rather than computed, which serves as a no-computation control.

are a fixed random function of the variable’s *name* (Hewitt and Liang, 2019); where the control also scores highly, the probe is reading the identity of the name rather than a value.

Patching. To test whether the name causes a behavior, we replace the activations at its tokens with those of the matched twin and read the output again (Zhang and Nanda, 2024). Two controls accompany every intervention: replacing one neutral word with another bounds the damage caused by any replacement, and replacing one lure with a different lure indicates whether the answer follows the name.

Models. Gemma-3-12B-it, Gemma-3-4B-it, Llama-3.1-8B, Llama-3.1-8B-Instruct, Llama-3.2-3B, Llama-3.2-3B-Instruct, OLMo-2-1B-Instruct, OLMo-2-7B-Instruct.

3. Results

Replication. On the letter condition, our per-token probes reproduce the results of Kudo et al. for Llama-3.2-3B: the queried variable becomes decodable at equation 5 of the chain and the intermediate variable at equation 2, with pre-CoT maxima of 17.1 and 47.9 against their 17.8 and 33.2.

Behavior. Table 3 reports accuracy and the pre-registered contrasts for each model and regime, averaged over three demonstration sets. Without CoT, the models use the value denoted by the name. A congruent name on the queried variable raises the accuracy of Llama-3.1-8B by +4.8 points at level 3 and by up to 20 points at level 5, and an incongruent name is answered +1.2 points above chance at level 3 and up to 20 points where accuracy is near the floor. With CoT, these contrasts are absent. An equivalence test with a margin of 2 points finds 29 of 30 CoT cells equivalent to zero, against 8 of 30 direct cells in which an effect is present. The one exception is the smallest model, OLMo-2-1B-Instruct, whose CoT solves 47% of level-3 problems with word names and 77% with letters. There, a lure on the intermediate variable adds +3.3 points [+2.5, +4.1].

Does the written value explain the effect of CoT? A natural explanation is that the chain states each variable’s value, and that the stated value replaces the value denoted by the name. We split the incongruent instances by whether the model’s own chain stated the true value of the target variable, and compare each half with the matched neutral twin passed through the same split. The raw lure rate differs considerably between the halves, but so does the rate at which the twin produces the same digit, because a model whose chain has broken down produces an arbitrary digit. After subtracting this baseline, the excess is +0.1 points over 102 cells in which the value was written and +0.1 over 34 cells in which it was not. The exception survives the control as well: OLMo-2-1B-Instruct is lured +3.7 points [+2.2, +5.2] on the 1207 instances in which its chain wrote the correct value.

Irrelevant lures. The irrelevant-lure condition at level 4 separates the presence of a number word from a conflict at a variable that is used. A number word on the distractor reduces accuracy but does not produce lure answers.

Model	Regime	Neutral accuracy (%)	number word on the queried variable		number word on the intermediate variable	
			helps	lures	helps	lures
Gemma-3-12B-it	CoT	100 [100,100]	+0.0	+0.0	+0.0	+0.0
Gemma-3-12B-it	direct	78 [76,81]	-3.2*	-0.2	-3.7*	+0.4
Gemma-3-4B-it	CoT	100 [100,100]	-0.0	+0.0	-0.0	+0.0
Gemma-3-4B-it	direct	55 [48,59]	+0.2	+0.9*	+0.6	+0.3
Llama-3.1-8B-Instruct	CoT	100 [99,100]	+0.2	+0.0	+0.3	+0.0
Llama-3.1-8B-Instruct	direct	52 [45,65]	+1.7	+0.1	+5.2*	+0.2
Llama-3.1-8B	CoT	95 [86,100]	-1.3	-0.1	+3.8	-0.0
Llama-3.1-8B	direct	53 [41,62]	+4.8*	+1.2*	+3.8*	-0.0
Llama-3.2-3B-Instruct	CoT	100 [100,100]	-0.0	+0.0	-0.2	+0.0
Llama-3.2-3B-Instruct	direct	44 [40,50]	-1.8*	+0.2	+0.5	+0.5
Llama-3.2-3B	CoT	98 [97,100]	-1.2	+0.1	+0.9	-0.2*
Llama-3.2-3B	direct	22 [13,31]	+5.4*	+2.8*	+1.1	+1.8
OLMo-2-1B-Instruct	CoT	47 [44,51]	-5.8*	+0.5	+6.9*	+3.3*
OLMo-2-1B-Instruct	direct	34 [30,38]	-0.0	+2.7*	+3.0*	-0.2
OLMo-2-7B-Instruct	CoT	95 [93,97]	+0.6*	-0.1	-2.7	-0.0
OLMo-2-7B-Instruct	direct	54 [50,58]	+0.6	-0.2	+3.8*	-0.4

Table 3: Does the name’s value change what the model answers? Level 3, averaged over three demonstration sets, with the range across those sets in brackets. *Helps* is the accuracy gained when the number word matches the variable’s value (congruent minus neutral, in points). *Lures* is how much more often the model answers the number word’s own value than it would by chance (incongruent minus the rate at which the neutral twin gives that same digit). * marks an effect whose sign is the same in all three demonstration sets and whose bootstrap interval excludes zero. Every starred effect is in the direct rows. Other levels are in Table 4.

Word class. One effect is not removed by CoT. At level 4, a number word on the queried variable reduces the accuracy of Llama-3.2-3B by 9.9 points whether or not the denoted value agrees with the true value; in these instances the model restates the wrong equation in the following step.

Probing results. Figure 1(c) shows the probe predictions at each token, and Figure 2 in the appendix the corresponding layer sweep. At P2, the end of the defining equation, the true value is partly computed (accuracy 0.49, selectivity 0.32) and the lure value is weakly decodable (0.06). At P3, the query, a probe reads the lure value in 32% of instances, but the control task also scores 0.97 there, so the probe is reading the identity of the name. At P4 and P5, the true value is decodable at 0.98 and the lure value is at zero in every condition and level. A number word on a computed variable has a lure mass of 0.007 at the answer position, against 0.118 for one on the distractor, which is never computed.

Causal intervention. Figure 3 in the appendix shows, for the direct regime, the proportion of lure errors removed by patching the tokens of the name from the matched neutral twin, by layer, with both controls. For Llama-3.2-3B, the neutral patch removes 52% of the lure errors and restores the mar-

gin between the true and lure values (normalized logit difference 1.00), the alternative-lure patch is followed in 9% of cases, and the span grid places the effect at the equation that defines the name, at layers 0–7. Because the word control changes 22% of the correct answers at this floor accuracy, we report the localization rather than a recovery rate. For Llama-3.1-8B, the no-CoT lure rate at level 3 is at chance and the two patches behave alike. With CoT, there are no lure errors to remove, and injecting the activations of the name into a neutral run does not change the answer in any instance.

4 Related Work

Kudo et al. (2026) established when sub-answers are computed during CoT on clean arithmetic, and we add a cue conflict to that setting. Two studies apply the Stroop design to language models: Wang (2026) place a local redefinition of a word in conflict with its familiar associate, and Hu et al. (2026) measure a congruency effect in next-token probabilities on a color-rule task. Both measure a single next token, and neither involves errors, a chain of thought, or probes over reasoning steps. Faithfulness has also been measured externally, by perturbing prompts, chains, or parameters (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025; Tutek et al., 2025), whereas we measure it internally under a cue for which both values are known. Since small models can copy the trailing number of the chain (Liu, 2026) and an output prior can override a correct chain (Prabhakar et al., 2024), we report a positional-copy control. Misleading identifiers degrade performance on real code (Le et al., 2026, 2025; Lam et al., 2025), whereas our task is synthetic, which allows the effect to be localized. Work on binding asks how a value is attached to a name (Feng and Steinhardt, 2024; Wu et al., 2025; Gur-Arieh et al., 2026), and we ask whether that attachment survives a competing value.

5 Conclusion

We introduced a cue-conflict version of a controlled arithmetic task in order to ask where a misleading variable name interferes. Without CoT, the value denoted by the name is used as the answer, and patching localizes this effect to the early layers at the equation that defines the name. With CoT, we observe no effect at any level, with one exception among eight models, and whether the

chain states the value does not predict the size of the effect.

Limitations

The task is synthetic arithmetic with short chains, so the results need not transfer to real code, which is the motivation for the task rather than a finding. The models are at most 8B parameters, and within-family comparisons indicate the direction of a trend only. The lures are English number words; digit-bearing names such as x2, other languages, and optimized lures remain future work. Linear probes can miss information that is encoded nonlinearly; the patching experiments address this in part, but hidden states carry mixed information and patching cannot exclude side effects, which is why we report two control patches. Finally, the CoT format writes the name beside the computed value at every step, which may itself assist recovery; the direct regime bounds this effect from the other side.

References

Yanda Chen et al. 2025. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*.

Jiahai Feng and Jacob Steinhardt. 2024. How do language models bind entities in context? In *International Conference on Learning Representations*.

Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.

Yoav Gur-Arieh, Mor Geva, and Atticus Geiger. 2026. Mixing mechanisms: How language models retrieve bound entities in-context. In *International Conference on Learning Representations*. ArXiv:2510.06182.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.

Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.

Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38. ArXiv:2504.14119.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiušė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.

Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*. Cite in the third person under [review].

Ming Liu. 2026. The readout shortcut: Positional number copying dominates arithmetic CoT readout in small language models. *arXiv preprint arXiv:2605.22870*.

Akshara Prabhakar, Thomas L. Griffiths, and R. Thomas McCoy. 2024. Deciphering the factors influencing the efficacy of chain-of-thought: Probability, memorization, and noisy reasoning. *arXiv preprint arXiv:2407.01687*. Venue to verify (likely Findings of EMNLP 2024).

J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Martin Tutek, Fateme Hashemi Chaleshtori, Ana Marasović, and Yonatan Belinkov. 2025. Measuring chain of thought faithfulness by unlearning reasoning steps. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Han-yu Wang. 2026. Persistent priors, preserved targets: A Stroop-style paradigm for lexical override. *arXiv preprint arXiv:2606.07555*.

Yiwei Wu, Atticus Geiger, and Raphaël Millière. 2025. How do transformers learn variable binding in symbolic programs? In *Proceedings of the 42nd International Conference on Machine Learning*.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Tokenizer checks

Rule 4 is decided per model, not by eye. For each model we tokenize every candidate name in the five contexts of the template (line start, after a comma, after each operator, before the separator) and keep only words that are a single token in all five with no merging into a neighbour. For the Llama 3 tokenizer, all ten number words pass and 95 of 112 neutral candidates pass; the lists are released with the code. Binary operators carry a space on each side because – otherwise merges with a following word (1-two tokenizes as 1, -two), which would break the alignment between twins. With the released lists, every instance of a condition tokenizes to the same length with the same position indices, and the runner excludes and counts any instance that does not; no instance was excluded in any run reported here.

B Full sweeps and selectivity

For every (variable, position, layer, seed) we report probe accuracy for the true value, lure rate, lure mass and margin, alongside the accuracy of a control probe trained on labels that are a deterministic random function of the variable’s name (Hewitt and Liang, 2019). Selectivity is the difference. The query position is the one cell where a high lure rate coincides with control accuracy near 1, which is why it is reported as name identity rather than as a represented value. Per-token sweeps over the whole instance, with the max-over-layers curve, are given for both regimes and all three conditions, together with t^* , t_{eq}^* and the pre-/post-chain accuracies in Kudo et al.’s format.

C Additional levels, models and demonstration sets

Table 4 repeats Table 3 for the other four levels.

Model	Level	Regime	Neutral accuracy (%)	number word on the queried variable		number word on the intermediate variable	
				helps	lures	helps	lures
Llama-3.1-8B	1	CoT	99 [98,100]	+0.0	+0.0	+0.3	+0.0
Llama-3.1-8B	1	direct	74 [38,99]	+16.1*	+11.2*	+8.8*	+1.3
Llama-3.2-3B	1	CoT	97 [95,100]	+1.1	+0.1	-2.4	+0.3
Llama-3.2-3B	1	direct	46 [21,93]	+0.6	+5.9*	-3.9*	+2.6*
Llama-3.1-8B	2	CoT	100 [100,100]	-0.1	+0.0	-0.0	+0.0
Llama-3.1-8B	2	direct	36 [33,37]	+11.8*	+0.7	+2.6	+2.2
Llama-3.2-3B	2	CoT	94 [90,98]	+3.7	+0.5*	+1.0*	-0.1
Llama-3.2-3B	2	direct	29 [27,30]	+10.8*	+2.5*	+2.6*	+2.2
Llama-3.1-8B	4	CoT	88 [63,100]	+1.0	-0.0	-2.3	+0.2
Llama-3.1-8B	4	direct	28 [26,31]	+13.7*	+5.8	+6.0*	+0.2
Llama-3.2-3B	4	CoT	64 [26,85]	-10.2*	+0.7	-2.6*	+0.4
Llama-3.2-3B	4	direct	20 [16,23]	-1.6	+1.9	-0.2	+1.2*
Llama-3.1-8B	5	CoT	65 [43,100]	+1.2	-0.3	+0.1	-0.1
Llama-3.1-8B	5	direct	20 [13,26]	+20.2*	+20.5*	+0.9	+0.4
Llama-3.2-3B	5	CoT	69 [48,97]	-1.4	+0.1	-2.5	-0.1
Llama-3.2-3B	5	direct	12 [11,12]	+4.7*	+3.5*	+1.1*	+0.5

Table 4: As Table 3, for levels 1, 2, 4 and 5. Level 1 is the no-computation control: the intermediate variable’s value is stated rather than computed.

Per-set values, the pooled estimate and the claim rule are released with the code. The appendix also contains the span-by-layer patching grids for all three sources, the level-4 irrelevant-lure control, the lure-distance analysis, the teacher-forced lure rates, and the error taxonomy that identifies the equation-selection failure behind the word-class effect.

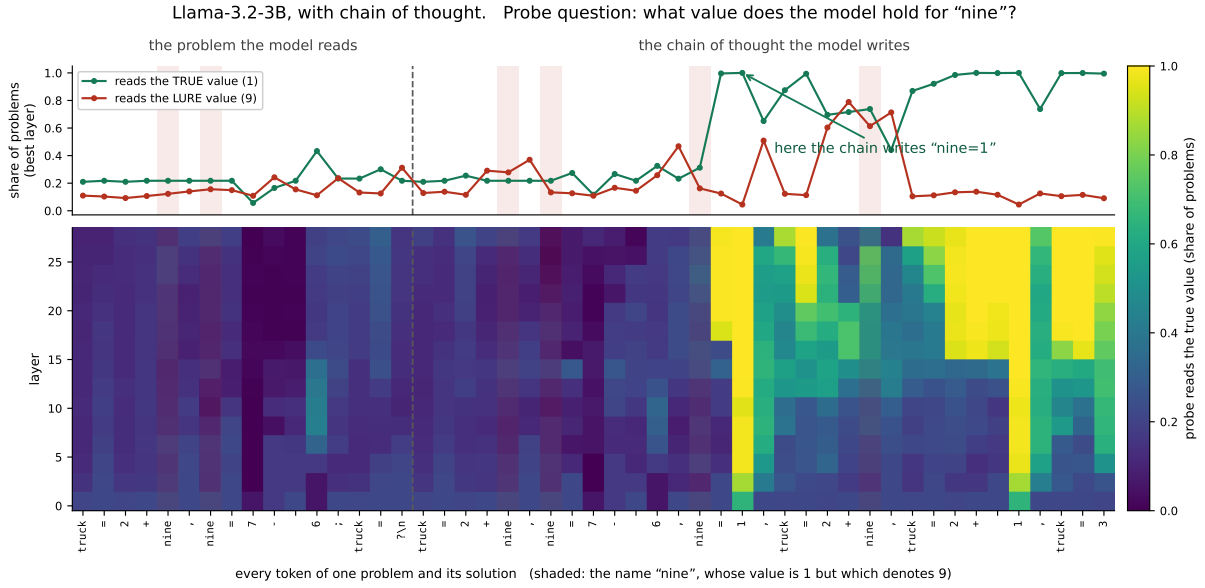


Figure 2: What the model holds for the variable whose name lies. The probe is asked, at every token, for the value of nine, a variable whose name denotes 9 but whose value is 1. **Top:** how often the probe’s answer is the true value (green) and how often it is the lure (red), taking the best layer at each token. **Bottom:** the same true-value accuracy for every layer. The problem is to the left of the dashed line and the model’s own chain of thought to the right; shaded columns are the tokens of the name itself. The lure is read only at those tokens and at the query, and it disappears the moment the chain writes `nine=1`, after which the true value is held for the rest of the solution. One problem’s tokens are shown; positions are the same for all 2,000.

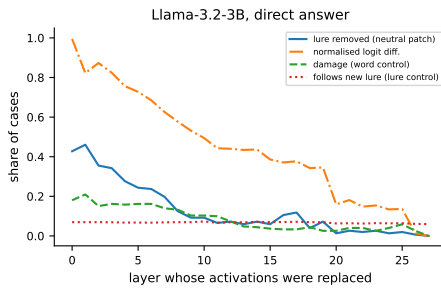


Figure 3: Lure errors removed by patching the name’s tokens from the neutral twin, by layer, direct regime, with the neutral-word control patch (damage) and the alternative-lure patch.